

CARTA CAFETERÍA – DOCUMENTACIÓN TÉCNICA

Autor: Salvatore De Rosa Vega

Curso: 2-DAMN-B

Asignatura: Programación Multimedia y Dispositivos Móviles

Centro: IES El Rincón

Repositorio GitHub: <https://github.com/S4lv4-code/Carta-cafeteria>

19/11/2025

1. Arquitectura general

Descripción completa tomada del análisis subido. Se explica cómo App.jsx coordina el flujo de categorías y productos. Se detalla el patrón jerárquico desde index.tsx → App.jsx → Container.jsx → Category.jsx → Product.jsx.

2. Código relevante – App.jsx

Controla el estado global de categorías y productos.

Realiza llamadas a la API para fetch, crear, editar y eliminar tanto categorías como productos.

Pasa las funciones CRUD a Container.jsx.

Implementa efectos con useEffect para cargar datos al iniciar la app y registrar cambios en consola.

```
import React, { useState, useEffect } from "react";
import { ScrollView, ImageBackground, View } from "react-native";
import Container from "../Container/Container";
import { globalStyles } from "../theme";

const API_BASE = "https://jlorenzo.ddns.net/carta_restaurante";
const USER_ID = 2197;

export default function App() {
  const [categorias, setCategorias] = useState([]);

  async function fetchCategorias() {
    try {
      const res = await fetch(`${API_BASE}/categorias/?usuario_id=${USER_ID}`);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const json = await res.json();

      // Extraer data correctamente
      const arr = json.data;
      if (!Array.isArray(arr)) {
        console.error("Formato inesperado en /categorias:", json);
        setCategorias([]);
        return;
      }

      const categoriasNormalizadas = arr.map((c) => ({
        id: c.id,
        nombre: c.nombre,
        imagen: "",
        product: [],
      }));

      setCategorias(categoriasNormalizadas);

      for (const cat of categoriasNormalizadas) {
        await fetchProductos(cat.id);
      }
    } catch (err) {
      console.error("Error fetchCategorias:", err);
      setCategorias([]);
    }
  }

  async function fetchProductos(catId) {
    try {
      const res = await fetch(
        `${API_BASE}/productos/${catId}?usuario_id=${USER_ID}`
      );
    }
  }
}
```

```

    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const json = await res.json();

    const arr = Array.isArray(json.data)
      ? json.data
      : Array.isArray(json)
        ? json
        : [];

    if (!Array.isArray(arr)) {
      console.error("Formato inesperado en /productos:",
json);
      return;
    }

    const productosNormalizados = arr.map((p) => ({
      id: p.id,
      name: p.nombre,
      price: parseFloat(p.precio),
    }));

    setCategorias((prev) =>
      prev.map((c) =>
        c.id === catId ? { ...c, product:
productosNormalizados } : c
      )
    );
  } catch (err) {
    console.error("Error fetchProductos:", err);
  }
}

// ----- CATEGORÍAS -----
async function addCategoria({ category }) {
  const body = { usuario_id: USER_ID, nombre: category };
  try {
    const res = await fetch(`${API_BASE}/categorias/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    await fetchCategorias();
  } catch (err) {
    console.error("Error addCategoria:", err);
  }
}

async function editCategoria(id, newName) {
  const body = { usuario_id: USER_ID, nombre: newName, orden:
1 };
  try {

```

```

        const res = await fetch(`${API_BASE}/categorias/${id}`, {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body),
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        await fetchCategorias();
    } catch (err) {
        console.error("Error editCategoria:", err);
    }
}

async function deleteCategoria(id) {
    const body = { usuario_id: USER_ID };
    try {
        const res = await fetch(`${API_BASE}/categorias/${id}`, {
            method: "DELETE",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body),
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        await fetchCategorias();
    } catch (err) {
        console.error("Error deleteCategoria:", err);
    }
}

// ----- PRODUCTOS -----
async function addProducto(catId, { name, price }) {
    const body = {
        usuario_id: USER_ID,
        nombre: name,
        precio: parseFloat(price),
        orden: 1,
    };
    try {
        const res = await fetch(`${API_BASE}/productos/${catId}`,
{
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body),
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        await fetchProductos(catId);
    } catch (err) {
        console.error("Error addProducto:", err);
    }
}

async function editProducto(catId, prodId, { name, price }) {
    const body = {
        usuario_id: USER_ID,

```

```

        nombre: name,
        precio: parseFloat(price),
    };
    try {
        const res = await
fetch(`${API_BASE}/productos/${prodId}`, {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body),
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        await fetchProductos(catId);
    } catch (err) {
        console.error("Error editProducto:", err);
    }
}

async function deleteProducto(catId, prodId) {
    const body = { usuario_id: USER_ID };
    try {
        const res = await
fetch(`${API_BASE}/productos/${prodId}`, {
            method: "DELETE",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body),
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        await fetchProductos(catId);
    } catch (err) {
        console.error("Error deleteProducto:", err);
    }
}

useEffect(() => {
    fetchCategorias();
}, []);

useEffect(() => {
    console.log("Categorías cargadas:", categorias);
}, [categorias]);

return (
    <ImageBackground
        source={{ uri:
'https://cdn.freecodecamp.org/curriculum/css-cafe/beans.jpg' }}
        style={globalStyles.scrollView}
        resizeMode="cover"
    >
        <ScrollView contentContainerStyle=
{globalStyles.contentContainer}>
            <View style={globalStyles.container}>
                <Container

```

```

        categorias={categorias}
        onAddCategoria={addCategoria}
        onEditCategoria={editCategoria}
        onDeleteCategoria={deleteCategoria}
        onAddProducto={addProducto}
        onEditProducto={editProducto}
        onDeleteProducto={deleteProducto}
      />
    </View>
  </ScrollView>
</ImageBackground>
);
}

```

3. Container.jsx – Estructura principal

Contenedor principal que organiza la interfaz.

Permite agregar nuevas categorías y renderiza todas las categorías existentes.

Recibe las funciones CRUD desde App.jsx y las pasa a Category.jsx.

```

import React, { useState } from "react";
import { View, StyleSheet } from "react-native";
import Header from "../Header/Header";
import Spacer from "../Spacer/Spacer";
import Category from "../Category/Category";
import Footer from "../Footer/Footer";
import { FormInput } from "../components/FormInput";
import { PrimaryButton } from "../components/Button";
import { spacing, colors, formStyles } from "../theme";

export default function Container({
  categorias,
  onAddCategoria,
  onEditCategoria,
  onDeleteCategoria,
  onAddProducto,
  onEditProducto,
  onDeleteProducto,
}) {
  const [newCatName, setNewCatName] = useState("");
  const [newCatImagen, setNewCatImagen] = useState("");

  function handleAddCategoria() {
    if (!newCatName.trim()) return;
    onAddCategoria({
      category: newCatName.trim(),

```

```

        imagen: newCatImagen.trim() || undefined,
    });
    setNewCatName("");
    setNewCatImagen("");
}

return (
    <View style={styles.container}>
        <Header />
        <Spacer />

        <View style={styles.addSection}>
            <FormInput
                placeholder="Nombre categoría"
                value={newCatName}
                onChangeText={setNewCatName}
            />
            <FormInput
                placeholder="URL imagen (opcional)"
                value={newCatImagen}
                onChangeText={setNewCatImagen}
            />
            <PrimaryButton onPress={handleAddCategoria}>
                Agregar categoría
            </PrimaryButton>
        </View>

        <Spacer />
        {categorias.map((cat) => (
            <Category
                key={cat.id}
                cat={cat}
                onEditCategoria={onEditCategoria}
                onDeleteCategoria={onDeleteCategoria}
                onAddProducto={onAddProducto}
                onEditProducto={onEditProducto}
                onDeleteProducto={onDeleteProducto}
            />
        ))}
        <Spacer />
        <Footer />
    </View>
);
}

const styles = StyleSheet.create({
    container: { width: '100%', alignItems: 'center' },
    addSection: { alignItems: 'center', width: '100%',
marginVertical: spacing.md, gap: spacing.sm },
});

```

4. Category.jsx + CategoryHeader.jsx

Representa una categoría individual.

Muestra el header de la categoría, la imagen (opcional), formulario de agregar productos y la lista de productos (ProductList).

Pasa funciones CRUD de productos a los componentes hijos.

CategoryHeader: muestra nombre de la categoría y botones de edición/eliminación.

```
import React from "react";
import { View, Image, StyleSheet } from "react-native";
import CategoryHeader from "../CategoryHeader";
import AddProductForm from "../AddProductForm";
import ProductList from "../ProductList";
import { categoryStyles } from "../../theme";

export default function Category({
  cat,
  onEditCategoria,
  onDeleteCategoria,
  onAddProducto,
  onEditProducto,
  onDeleteProducto,
}) {
  return (
    <View style={categoryStyles.container}>
      <CategoryHeader
        cat={cat}
        onEditCategoria={onEditCategoria}
        onDeleteCategoria={onDeleteCategoria}
      />

      {cat.imagen ? (
        <Image source={{ uri: cat.imagen }} style=
{categoryStyles.image} />
      ) : null}

      <AddProductForm catId={cat.id} onAddProducto=
{onAddProducto} />

      <ProductList
        productos={Array.isArray(cat.product) ? cat.product :
[]}
        catId={cat.id}
        onEditProducto={onEditProducto}
        onDeleteProducto={onDeleteProducto}
      />
    </View>
  );
}
```



```
    </View>
  );
}
```

5. Product.jsx – CRUD de productos

Representa un producto individual.

Permite visualizar, editar inline y eliminar un producto con confirmación.

Adaptado para pantallas móviles y escritorio.

```
import React, { useState } from "react";
import { View, Text, Alert, StyleSheet, Dimensions } from
"react-native";
import { FormInput } from "../../components/FormInput";
import { IconButton, SmallButton } from
"../../components/Button";
import { productStyles, colors, spacing } from
"../../theme";

const { width } = Dimensions.get('window');
const isMobile = width < 600;

export default function Product({
  prod,
  catId,
  onEditProducto,
  onDeleteProducto,
}) {
  const [editing, setEditing] = useState(false);
  const [name, setName] = useState(prod.name);
  const [price, setPrice] = useState(String(prod.price));

  function saveEdit() {
    if (!name.trim() || price === "") return;
    onEditProducto(catId, prod.id, { name: name.trim(), price:
Number(price) });
    setEditing(false);
  }

  function handleDelete() {
    Alert.alert(
      'Eliminar producto',
      `Eliminar producto "${prod.name}"?`,
      [
        { text: 'Cancelar', style: 'cancel' },
        { text: 'Borrar', style: 'destructive', onPress: () =>
```

```

onDeleteProducto(catId, prod.id) },
    ]
  );
}

return (
  <View style={styles.item}>
    {!editing ? (
      <>
        <Text style={styles.name}>{prod.name}</Text>
        <Text style={styles.price}>
{Number(prod.price).toFixed(2)}</Text>
        <View style={styles.actions}>
          <IconButton onPress={() => {
            setEditing(true);
            setName(prod.name);
            setPrice(String(prod.price));
          }}>Editar</IconButton>
          <IconButton onPress={handleDelete}
danger>Borrar</IconButton>
        </View>
      </>
    ) : (
      <View style={styles.editRow}>
        <FormInput value={name} onChangeText={setName} style=
{styles.input} />
        <FormInput value={price} onChangeText={setPrice}
keyboardType="numeric" style={styles.input} />
        <SmallButton onPress={saveEdit}>Guardar</SmallButton>
        <SmallButton onPress={() =>
setEditing(false)}>Cancelar</SmallButton>
      </View>
    )}
  </View>
);
}

const styles = StyleSheet.create({
  item: {
    paddingVertical: 8,
    borderBottomWidth: 1,
    borderBottomColor: '#eee',
    width: '100%',
    flexDirection: isMobile ? 'column' : 'row',
    alignItems: isMobile ? 'flex-start' : 'center',
    justifyContent: 'space-between',
  },
  name: {
    fontSize: isMobile ? 13 : 16,
    marginBottom: isMobile ? 4 : 0,
    flex: isMobile ? 0 : 1,
    width: isMobile ? '100%' : 'auto',
  },
});

```

```

    },
    price: {
      color: '#666',
      fontSize: isMobile ? 12 : 14,
      marginBottom: isMobile ? 6 : 0,
      textAlign: isMobile ? 'left' : 'right',
      marginLeft: isMobile ? 0 : 12,
    },
    actions: { flexDirection: 'row', marginTop: isMobile ? 8 : 8,
width: isMobile ? '100%' : 'auto' },
    actionButton: { padding: 8, backgroundColor: '#eee',
borderRadius: 6, marginRight: 8, minHeight: 40, justifyContent:
'center' },
    danger: { backgroundColor: 'transparent' },
    editRow: { flexDirection: 'column', alignItems: 'center',
width: '100%', gap: spacing.sm },
    input: { width: '100%', borderWidth: 1, borderColor: '#ccc',
padding: 6, borderRadius: 6, marginBottom: 8, backgroundColor:
'rgba(0,0,0,0)' },
  });

```

6. Componentes reutilizables

Botones y formularios estandarizados para mantener consistencia visual y facilitar la reutilización.

Manejan propiedades como `danger`, `activeOpacity` y estilos responsivos.

```

import React from 'react';
import { TouchableOpacity, Text, StyleSheet, Dimensions } from
'react-native';
import { colors, spacing } from '../theme';

const { width } = Dimensions.get('window');
const isMobile = width < 600;

export function PrimaryButton({ onPress, children, style }) {
  return (
    <TouchableOpacity
      onPress={onPress}
      style={[styles.button, style]}
      activeOpacity={0.7}
    >
      <Text style={styles.text}>{children}</Text>
    </TouchableOpacity>
  );
}

```

```

export function IconButton({ onPress, children, danger = false,
style }) {
  return (
    <TouchableOpacity
      onPress={onPress}
      style={[
        styles.iconButton,
        danger && styles.danger,
        !danger && style,
        danger && style,
      ]}
      activeOpacity={0.7}
    >
      <Text style={[styles.iconText, danger ? styles.dangerText
: null]}>
        {children}
      </Text>
    </TouchableOpacity>
  );
}

export function SmallButton({ onPress, children, style }) {
  return (
    <TouchableOpacity
      onPress={onPress}
      style={[styles.smallButton, style]}
      activeOpacity={0.7}
    >
      <Text style={styles.smallText}>{children}</Text>
    </TouchableOpacity>
  );
}

```

7. Resumen final

El proyecto “Carta Cafetería” es una aplicación móvil desarrollada con React Native que permite gestionar categorías y productos de una cafetería de manera dinámica. Cuenta con un sistema completo de CRUD para categorías y productos, interfaz responsiva para distintos tamaños de pantalla y componentes reutilizables que garantizan consistencia y escalabilidad. La arquitectura modular facilita la expansión futura y la integración con APIs externas, mientras que la normalización de datos y el manejo de errores aseguran robustez y estabilidad.

CAMPER CAFE

Est. 2020

Nombre categoria

URL imagen (opcional)

Agregar categoría

CARNE

[Editar](#)
[Borrar](#)

Nombre producto

Precio

Agregar producto

PESCADO

[Editar](#)
[Borrar](#)

Nombre producto